

Auditoría del repositorio

Alcance y método

Se inspeccionaron todos los archivos versionados, las fuentes Python, los datos web, la interfaz, las pruebas, los planes, los PDF existentes y los recursos visuales. La auditoría distingue entre fuente editable, artefacto generado y estado guardado en el navegador. No se modificó la lógica de la aplicación.

Resultado ejecutivo

THM Fieldbook es una PWA estática en JavaScript nativo. Su contenido educativo no procede de una base de datos ni de una API: se compone en Python, se exporta a `web/data/content.json` y se consume en el navegador. La base actual es amplia como playbook: 24 secciones, 441 comandos, 88 conceptos, 19 rutas, 12 guías y 215 fichas de comandos. Sin embargo, no tiene todavía la estructura pedagógica, los fundamentos, las evaluaciones ni la separación entre teoría, práctica y soluciones que exige un curso.

Estructura relevante

Ruta	Función	Naturaleza
tools/generate_structured_playbook.py	Define SECTIONS, bloques, tablas y comandos del playbook estructurado; genera PDF con ReportLab/PyPDF	Fuente primaria y generador
tools/generate_thm_playbook.py	Define GENERATED_DOCS y genera el playbook maestro y guías PDF históricas	Fuente primaria heredada y generador
tools/concepts.py	Define 88 conceptos y 19 rutas de aprendizaje	Fuente primaria pedagógica
tools/guides.py	Define 12 guías secuenciales	Fuente primaria metodológica
tools/command_metadata.py	Añade objetivo, requisitos, salida, riesgo, errores y alternativas a 215 comandos	Fuente primaria de contexto operativo
tools/export_web_content.py	Normaliza y combina las fuentes anteriores	Transformación reproducible
web/data/content.json	Exportación con secciones, conceptos, rutas, guías y metadatos	Artefacto generado; no editar a mano
web/data/revshells.json	11 plantillas de reverse shell y 5 pasos de estabilización	Fuente de datos independiente
web/app.mjs	Presentación, búsqueda, adaptación de variables, navegación y estado	Lógica de aplicación; contiene también catálogo de puertos y ayudas puntuales
web/js/*.mjs	Búsqueda, intención, progreso, rooms y generación de informe	Lógica modular
web/index.html, styles.css, icon.svg	Estructura y presentación visual	Interfaz y activo reutilizable
PT1_*.md, PLAN_*.md, GUIA_DE_USO.md	Cobertura, decisiones y uso previsto	Contexto editorial y de alcance
output/pdf/	PDF históricos generados	Artefactos, no fuente
output/*.png	Capturas de la PWA	Recursos visuales reutilizables con finalidad ilustrativa
tests/, web/*.test.mjs, web/js/*.test.mjs	Validación del exportador y la aplicación	Control de calidad

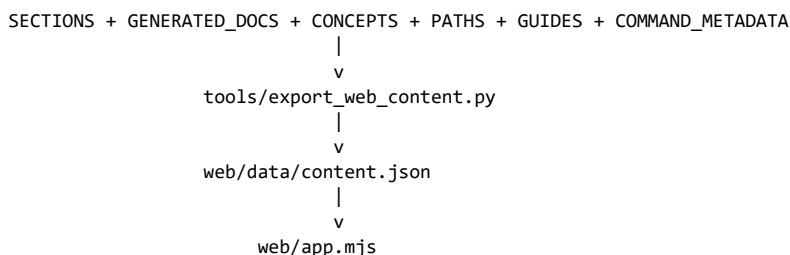
No se encontraron seeders, migraciones, fixtures de negocio, backend, API propia ni base de datos local. El estado de rooms, favoritos y progreso se conserva mediante `localStorage` y `sessionStorage`; no es contenido curricular.

Tecnologías

- HTML5, CSS y módulos ECMAScript, sin framework de interfaz.
- PWA con `manifest.webmanifest` y `service worker`.
- Python 3 para modelado, exportación y generación de PDF.
- ReportLab y PyPDF para los PDF existentes.

- JSON como formato de entrega a la interfaz.
- `unittest` para el exportador y `node:test` para la aplicación.
- GitHub Pages como despliegue estático.

Flujo real del contenido



`content.json` es la fuente de ejecución de la PWA, pero no la fuente editorial. Los documentos del curso deben citar las fuentes Python correspondientes y tratar el JSON solo como prueba del resultado exportado.

Categorías encontradas

Categoría	Evidencia principal	Cobertura actual
Metodología y workflow	7 secciones de inicio, atasco, errores y cierre; guía metodología	Fuerte como procedimiento, parcial como teoría
Reconocimiento y servicios	Nmap, DNS, FTP, SMB, SMTP, SNMP, NFS, LDAP y bases de datos	Fuerte
Web y APIs	Discovery, autorización, inyecciones, ficheros, WordPress y SQLi	Fuerte en técnicas seleccionadas
Acceso, credenciales y cracking	Loot, hashes, reutilización, shells y transferencia	Fuerte
Linux	Enumeración y múltiples vectores de escalada	Fuerte
Windows	Enumeración, servicios, registro, privilegios, DLL y UAC	Fuerte
Active Directory	LDAP, Kerberos, roasting, PTH, BloodHound, AD CS, tickets, ACL y trusts	Media-alta
Redes y pivoting	Túneles, rutas, proxying, segmentación y diagnóstico	Fuerte
Explotación y cierre	CVE, Metasploit, evidencia, informe y limpieza	Media-alta

Contenido reutilizable

- Los campos explicativos de cada concepto: qué es, causa, contexto, señales, confirmación, resultado y pasos.
- Las rutas de aprendizaje, útiles para fijar prerrequisitos.
- Las guías, útiles para construir casos y flujos de decisión.
- Los metadatos de comandos, esenciales para explicar objetivo, precondiciones, evidencia y errores.
- Las tablas de `SECTIONS`, que resumen decisiones y síntomas.
- Las plantillas de notes/reporting y la plantilla PT1, reutilizables en metodología.

- Las capturas de la interfaz, opcionales para explicar la relación curso-playbook.

Problemas y carencias detectados

1. Los fundamentos de HTTP, parsing, DNS, redes, shell, bases de datos y flujo de datos aparecen dispersos o implícitos.
2. Muchos conceptos son excelentes fichas operativas, pero no cumplen aún la estructura didáctica requerida para un módulo autónomo.
3. Solo 215 de 441 comandos de sección tienen ficha extensa de metadatos. El resto conserva contexto en el bloque o la tabla, pero requiere revisión antes de enseñarse fuera de su sección.
4. Hay 611 apariciones de comandos al sumar secciones y conceptos, 534 cadenas únicas y 62 cadenas repetidas. Parte de la repetición es deliberada; sin trazabilidad produciría divergencias.
5. `content.json` duplica todo el contenido exportado. Editarlo manualmente rompería la fuente única.
6. Hay texto heredado con secuencias de mojibake en varias fuentes y en la interfaz. Debe distinguirse de acentos válidos antes de corregirlo; no se cambiará como parte del curso salvo que afecte a una cita.
7. Los PDF actuales se generan directamente desde estructuras Python, no desde Markdown. No satisfacen el nuevo requisito de que Markdown sea la fuente de verdad.
8. Faltan cuadernos, exámenes, rúbricas y solucionarios independientes.
9. No existe comprobador general de enlaces Markdown, esquema de metadatos documentales ni matriz de trazabilidad.
10. Algunos temas solicitados carecen de concepto dedicado: CSRF, deserialización, request smuggling/desync, WebSocket smuggling, race conditions, prototype pollution, NoSQLi, RFI, systemd, DCSync y Golden/Silver Tickets. No deben presentarse como cubiertos por la aplicación.

Posibles errores técnicos a revisar

- Comandos sensibles a versión: sintaxis y nombre de ejecutable de Impacket, NetExec, BloodHound y herramientas de pivoting deben contrastarse antes de publicarse como material estable.
- Los payloads que incluyen JSON, comillas, barras inversas o plantillas pueden alterarse al pasar por Markdown, shell y PDF; se necesitan pruebas de preservación literal.
- El service worker cachea los JSON por versión. Una exportación sin actualización coherente de versión puede mostrar contenido anterior.
- Los generadores de PDF históricos crean salidas correctas, pero su diseño no cubre encabezados, índices y concatenación desde Markdown.

Estas observaciones no demuestran que los comandos sean incorrectos; señalan los puntos que requieren verificación específica.

Riesgos de duplicación

Riesgo	Control propuesto
Mismo comando en sección, concepto y chuleta	Definir un identificador canónico y citar el origen
Teoría repetida en varios módulos	Centralizar fundamentos y enlazarlos como prerequisite
Solución visible junto al ejercicio	Separar físicamente <code>08_ejercicios</code> y <code>10_solucionarios</code>
JSON editado a mano	Validar que se regenera desde Python
PDF divergente del Markdown	Generar siempre desde Markdown y no versionar ediciones manuales
Chuleta convertida en segundo curso	Limitarla a decisión, comando, variables y evidencia

Propuesta de transformación

1. Mantener intacta la PWA como playbook.
2. Crear un currículo de 14 semanas con fundamentos antes de explotación.
3. Convertir cada técnica realmente cubierta en un módulo autónomo enlazado a sus fuentes.
4. Documentar las carencias antes de ampliar temas no presentes.
5. Separar teoría, payloads, playbook, ejercicios, exámenes, soluciones y chuletas.
6. Añadir un generador Markdown a PDF independiente de la aplicación.
7. Validar metadatos, enlaces, bloques de código, referencias internas y PDF.

Línea base de calidad

Ejecutada el 2026-07-14:

```
python -m unittest discover -s tests -v      20/20 OK
node --test web/app.test.mjs web/js/*.test.mjs 7/7 OK
git diff --check      sin errores
```